binary_indexed_tree.cpp

```cpp
#pragma once
#include <bits/stdc++.h>
using namespace std;
template <typename T>
class BIT {
  int size;
  vector<T> dat;

public:
  BIT() {}
  BIT(int n) { init(n); }
  void init(int n) {
    size = n;
    dat.assign(size + 1, 0);
  }
  void add(int i, T x) {
    i++;
    while (i <= size) {
      dat[i] += x;
      i += i & -i;
    }
  }
  //[l,r)
  void add(int l, int r, T x) {
    add(l, x);
    add(r, -x);
  }
  //[0,i]
  T query(int i) {
    i++;
    T sum = 0;
    while (i > 0) {
      sum += dat[i];
      i -= i & -i;
    }
    return sum;
  }
  //[l,r)
  T query(int l, int r) {
    return query(r - 1) - query(l - 1);
  }
  //[0,x] >= w
  int lower_bound(T w) {
    if (w <= 0)
      return 0;
    int x = 0, k = 1;
    while ((k << 1) < size)
      k <<= 1;
    for (; k > 0; k >>= 1) {
      if (x + k <= size && dat[x + k] < w) {
        w -= dat[x + k];
        x += k;
      }
    }
    return x;
  }
}; //size, 0-indexed
```

binarytrie.cpp

```cpp
template <int B = 30>
struct
  binary_trie { //0 以上 2^B 未満の整数を管理する Binary Trie
  struct N {
    int nx[2] = {-1, -1}, cnt = 0;
  };
  vector<N> t{N()};

  void add(long long x,
    int d = 1) { //x の個数を d だけ増減する
    int v = 0;
    t[v].cnt += d;
    for (int i = B - 1; i >= 0; i--) {
      int b = x >> i & 1;
      if (t[v].nx[b] < 0)
        t[v].nx[b] = t.size(),
        t.emplace_back();
      v = t[v].nx[b];
      t[v].cnt += d;
    }
  }

  void insert(
    long long x) { //x を 1 個追加する
    add(x, 1);
  }

  void erase(long long x) { //x を 1 個削除する
    add(x, -1);
  }

  long long min_xor(long long
      x) { //集合内の y について x xor y の最小値を返す
    int v = 0;
    long long r = 0;
    for (int i = B - 1; i >= 0; i--) {
      int b = x >> i & 1, u = t[v].nx[b];
      if (u < 0 || !t[u].cnt)
        b ^= 1, r |= 1LL << i;
      v = t[v].nx[b];
    }
    return r;
  }

  long long max_xor(long long
      x) { //集合内の y について x xor y の最大値を返す
    int v = 0;
    long long r = 0;
    for (int i = B - 1; i >= 0; i--) {
      int b = x >> i & 1,
          u = t[v].nx[b ^ 1];
      if (u >= 0 && t[u].cnt)
        b ^= 1, r |= 1LL << i;
      v = t[v].nx[b];
    }
    return r;
  }
};
```

bmbm.cpp

```cpp
// modint-convolution が前提
// 数列 s から最短の線形漸化式
// s[n]=c[0]s[n-1]+...+c[d-1]s[n-d] を求める
vector<mint> BerlekampMassey(
  const vector<mint>& s) {
  vector<mint> c{1}, b{1};
  int l = 0, m = 1;
  mint d0 = 1;
  for (int n = 0; n < (int)s.size(); n++) {
    mint d = s[n];
    for (int i = 1; i <= l; i++)
      d += c[i] * s[n - i];
```

bmbm.cpp (cont.)

```
    if (!d.val()) {
      m++;
      continue;
    }
    auto t = c;
    mint z = d / d0;
    if (c.size() < b.size() + m)
      c.resize(b.size() + m);
    for (int i = 0; i < (int)b.size(); i++)
      c[i + m] -= z * b[i];
    if (2 * l <= n)
      l = n + 1 - l, b = t, d0 = d, m = 1;
    else
      m++;
  }
  c.erase(c.begin());
  for (auto& x : c)
    x = mint() - x;
  return c;
}
// 初項 a と漸化式 c から第 n 項を O(M(d)log
  ↪ n)で求める
mint BostanMori(vector<mint> a,
  vector<mint> c, long long n) {
  if (n < (long long)a.size())
    return a[n];
  int d = c.size();
  if (!d)
    return 0;
  a.resize(d);
  vector<mint> q(d + 1);
  q[0] = 1;
  for (int i = 0; i < d; i++)
    q[i + 1] = mint() - c[i];
  auto p = convolution(a, q);
  p.resize(d);
  while (n) {
    auto qm = q;
    for (int i = 1; i < (int)qm.size();
      i += 2)
      qm[i] = mint() - qm[i];
    auto x = convolution(p, qm),
         y = convolution(q, qm);
    vector<mint> np, nq;
    for (int i = n & 1; i < (int)x.size();
      i += 2)
      np.push_back(x[i]);
    for (int i = 0; i < (int)y.size();
      i += 2)
      nq.push_back(y[i]);
    p.swap(np);
    q.swap(nq);
    n >>= 1;
  }
  return p[0] / q[0];
}
// 数列の先頭部分 s から漸化式を推定し、第 n 項
  ↪ を求める
mint BMBM(
  const vector<mint>& s, long long n) {
  if (n < (long long)s.size())
    return s[n];
  return BostanMori(
    s, BerlekampMassey(s), n);
}
```

cartesiantree.cpp

```
template <class T>
struct CartesianTree {
  int root = 0; //最小値のうち index 最小が根
  vector<int> par, left, right;

  CartesianTree() {}
  CartesianTree(const vector<T>&
      a) // a の Cartesian Tree を O(n)で構築
      : par(a.size(), -1),
        left(a.size(), -1),
        right(a.size(), -1) {
    vector<int> st;
    for (int i = 0; i < (int)a.size();
      i++) {
      int p = -1;
      while (
        !st.empty() && a[st.back()] > a[i])
        p = st.back(), st.pop_back();
      if (!st.empty())
        par[i] = st.back(),
        right[st.back()] = i;
      if (p >= 0)
        par[p] = i, left[i] = p;
      st.push_back(i);
    }
    if (!st.empty())
      root = st[0];
  }
};
```

convex_hull_trick.cpp

```
template <class T, const T id>
struct convex_hull_trick {
  struct L {
    T a = 0, b = id;
    T f(T x) const {
      return a * x + b;
    } // 直線 ax+b の x での値
  };

  int n;
  vector<T> x;
  vector<L> t;
  vector<char> u;

  convex_hull_trick(const vector<T>&
      X) // 使用する x 座標を渡して構築
      : n(X.size()), x(X), t(4 * n),
        u(4 * n) {}

  void insert(
    T a, T b) { // 直線 ax+b を追加
    add(1, 0, n - 1, {a, b});
  }

  T get(T X)
    const { // x=X での追加済み直線の最大値
    int p =
      lower_bound(x.begin(), x.end(), X) -
      x.begin();
    assert(p < n && x[p] == X);
    return qry(1, 0, n - 1, p);
  }

private:
  void add(int k, int l, int r,
    L z) { // Li Chao Tree に直線 z を追加
```

convex_hull_trick.cpp (cont.)

```
    if (!u[k]) {
      t[k] = z;
      u[k] = 1;
      return;
    }
    int m = (l + r) / 2;
    bool a = z.f(x[l]) > t[k].f(x[l]);
    bool b = z.f(x[m]) > t[k].f(x[m]);
    if (b)
      swap(z, t[k]);
    if (l == r)
      return;
    if (a != b)
      add(2 * k, l, m, z);
    else
      add(2 * k + 1, m + 1, r, z);
  }

  T qry(int k, int l, int r,
    int p) const { // x[p]での最大値を求める
    if (!u[k])
      return id;
    T y = t[k].f(x[p]);
    if (l == r)
      return y;
    int m = (l + r) / 2;
    return max(y,
      p <= m ? qry(2 * k, l, m, p)
             : qry(2 * k + 1, m + 1, r, p));
  }
};
```

dsu.cpp

```
struct dsu {
  vector<int> p;
  dsu(int n) : p(n, -1) {}
  int leader(int x) {
    return p[x] < 0 ? x
                    : p[x] = leader(p[x]);
  }
  int merge(int a, int b) {
    a = leader(a);
    b = leader(b);
    if (a == b)
      return a;
    if (p[a] > p[b])
      swap(a, b);
    p[a] += p[b];
    p[b] = a;
    return a;
  }
  bool same(int a, int b) {
    return leader(a) == leader(b);
  }
  int size(int a) { return -p[leader(a)]; }
};
```

dynamic_segtree.cpp

```
template <class S, auto op, auto e>
class dynamic_segtree {
private:
  struct node;
  using node_ptr = unique_ptr<node>;
  struct node {
    long long idx;
    S value, product;
    node_ptr left, right;
    node(long long idx, S value)
```

dynamic_segtree.cpp (cont.)

```
        : idx(idx), value(value),
          product(value), left(nullptr),
          right(nullptr) {}

    void update() {
      product =
        op(op(left ? left->product : e(),
             value),
          right ? right->product : e());
    }
  };
  const long long n;
  node_ptr root;

public:
  dynamic_segtree() : dynamic_segtree(0) {}
  dynamic_segtree(long long n)
      : n(n), root(nullptr) {}

  void set(long long p, S x) {
    assert(0 <= p && p < n);
    set(root, 0, n, p, x);
  }
  void set(node_ptr& t, long long a,
    long long b, long long p, S x) {
    if (!t) {
      t = make_unique<node>(p, x);
      return;
    }
    if (t->idx == p) {
      t->value = x;
      t->update();
      return;
    }
    long long c = (a + b) / 2;
    if (p < c) {
      if (t->idx < p)
        swap(t->idx, p), swap(t->value, x);
      set(t->left, a, c, p, x);
    } else {
      if (p < t->idx)
        swap(p, t->idx), swap(x, t->value);
      set(t->right, c, b, p, x);
    }
    t->update();
  }

  S get(long long p) const {
    assert(0 <= p && p < n);
    return get(root, 0, n, p);
  }

  S get(const node_ptr& t, long long a,
    long long b, long long p) const {
    if (!t)
      return e();
    if (t->idx == p)
      return t->value;
    long long c = (a + b) / 2;
    if (p < c)
      return get(t->left, a, c, p);
    else
      return get(t->right, c, b, p);
  }

```

dynamic_segtree.cpp (cont.)

```
  S prod(long long l, long long r) {
    assert(0 <= l && l <= r && r <= n);
    return prod(root, 0, n, l, r);
  }

  S prod(const node_ptr& t, long long a,
    long long b, long long l,
    long long r) const {
    if (!t || b <= l || r <= a)
      return e();
    if (l <= a && b <= r)
      return t->product;
    long long c = (a + b) / 2;
    S result = prod(t->left, a, c, l, r);
    if (l <= t->idx && t->idx < r)
      result = op(result, t->value);
    return op(
      result, prod(t->right, c, b, l, r));
  }

  S all_prod() const {
    return root ? root->product : e();
  }
};
```

eulertour.cpp

```
struct EulerTour {
  int N;
  vector<int> vs, depth, id, id_depth,
    id_rev;
  vector<vector<int>> G;
  EulerTour(int n, vector<vector<int>>& g) {
    N = n;
    G = g;
    vs.resize(2 * N - 1); //DFS での訪問順
    depth.resize(
      2 * N - 1); // 訪問順での根からの深さ
    id.resize(
      N); //id[i]:i が初めて現れる index
    id_depth.resize(N); //頂点 i の深さ
    id_rev.resize(N); //i が最後に現れる index
    int k = 0;
    dfs(0, -1, 0, k);
  }
  void dfs(
    int now, int par, int d, int& k) {
    id[now] = k;
    id_rev[now] = k;
    id_depth[now] = d;
    vs[k] = now;
    depth[k++] = d;
    for (auto e : G[now]) {
      if (e == par)
        continue;
      dfs(e, now, d + 1, k);
      id_rev[now] = k;
      vs[k] = now;
      depth[k++] = d;
    }
  }
};
//int lca(int u,int v){ return vs[seg.prod(
  min(id[u],id[v]),max(id[u],id[v])+1).idx];
  }
```

geometry.cpp

```
using ld = long double;
using Point = complex<ld>;
```

geometry.cpp (cont.)

```
using pt = Point;
const ld EPS = 1e-10L, PI = acosl(-1.L);

inline bool equal(const ld& a,
  const ld& b) { //a,b が誤差を許して等しいか
  return fabsl(a - b) < EPS;
}
inline int sgn(
  const ld& x) { //x の符号を-1,0,1 で返す
  return (x > EPS) - (x < -EPS);
}
Point unitVector(const Point&
    a) { //a と同方向の単位ベクトル
  assert(abs(a) > EPS);
  return a / abs(a);
}
Point normalVector(const Point&
    a) { //a を反時計回りに 90 度回転
  return a * Point(0, 1);
}
ld dot(
  const Point& a, const Point& b) { //内積
  return real(conj(a) * b);
}
ld cross(
  const Point& a, const Point& b) { //外積
  return imag(conj(a) * b);
}
Point rotate(const Point& p,
  ld t) { //p を反時計回りに t(rad)回転
  return p * polar<ld>(1, t);
}
ld radianToDegree(ld x) { //rad を度に変換
  return x * 180 / PI;
}
ld degreeToRadian(ld x) { //度を rad に変換
  return x * PI / 180;
}

struct Line {
  Point a, b;
  Line() = default;
  Line(Point x, Point y)
      : a(x), b(y) {
  } //2 点 x,y を通る直線(線分)
  Line(
    ld A, ld B, ld C) { //Ax+By=C を表す直線
    assert(!equal(A, 0) || !equal(B, 0));
    if (!equal(B, 0))
      a = {0, C / B}, b = {1, (C - A) / B};
    else
      a = {C / A, 0}, b = {C / A, 1};
  }
};

struct Circle {
  Point p;
  ld r;
  Circle() = default;
  Circle(Point x, ld y)
      : p(x), r(y) {} //中心 x,半径 y の円
};

bool onLine(const Line& l,
  const Point& p) { //p が直線 l 上か
  return fabsl(cross(l.b - l.a, p - l.a)) <=
```

geometry.cpp (cont.)

```
    EPS;
}

bool onSegment(const Line& l,
  const Point& p) { //p が線分 l 上か
  return ccw(l.a, l.b, p) == 0;
}

Circle circumCircle(Point a, Point b,
  Point c) { //三角形 abc の外接円
  assert(fabsl(cross(b - a, c - a)) > EPS);
  Point x = (a + b) / 2, y = (a + c) / 2;
  Line l = {x, x + normalVector(b - a)};
  Line m = {y, y + normalVector(c - a)};
  Point p = crossPoint(l, m);
  return {p, abs(p - a)};
}

Point projection(const Line& l,
  const Point&
    p) { //l へ p から下ろした垂線の足
  Point d = l.b - l.a;
  assert(abs(d) > EPS);
  return l.a +
    d * dot(p - l.a, d) / norm(d);
}
Point reflection(const Line& l,
  const Point& p) { //l に関する p の対称点
  return 2.L * projection(l, p) - p;
}

int ccw(const Point& a, Point b,
  Point c) { //a,b,c の位置関係を判定
  b -= a;
  c -= a;
  ld x = cross(b, c);
  if (abs(x) > EPS)
    return x > 0 ? 1 : -1; //反時計/時計
  if (dot(b, c) < -EPS)
    return 2; //c-a-b
  if (norm(b) + EPS < norm(c))
    return -2; //a-b-c
  return 0; //c が線分 ab 上
}

bool isOrthogonal(const Line& a,
  const Line& b) { //2 直線が直交するか
  return equal(
    dot(a.b - a.a, b.b - b.a), 0);
}
bool isParallel(const Line& a,
  const Line& b) { //2 直線が平行か
  return equal(
    cross(a.b - a.a, b.b - b.a), 0);
}
bool isIntersect(const Line& a,
  const Line& b) { //2 線分が交差するか
  return ccw(a.a, a.b, b.a) *
      ccw(a.a, a.b, b.b) <=
    0 &&
    ccw(b.a, b.b, a.a) *
      ccw(b.a, b.b, a.b) <=
    0;
}

Point crossPoint(const Line& a,
  const Line& b) { //2 直線の交点
```

geometry.cpp (cont.)

```
  Point d = a.b - a.a, e = b.b - b.a;
  ld z = cross(d, e),
     w = cross(d, b.a - a.a);
  if (equal(z, 0)) {
    assert(equal(w, 0));
    return b.a;
  }
  return a.a + d * cross(b.a - a.a, e) / z;
}

ld distanceBetweenLineAndPoint(
  const Line& l,
  const Point& p) { //直線 l と点 p の距離
  Point d = l.b - l.a;
  assert(abs(d) > EPS);
  return fabsl(cross(d, p - l.a)) / abs(d);
}
ld distanceBetweenSegmentAndPoint(
  const Line& l,
  const Point& p) { //線分 l と点 p の距離
  Point d = l.b - l.a;
  assert(abs(d) > EPS);
  if (dot(d, p - l.a) < EPS)
    return abs(p - l.a);
  if (dot(-d, p - l.b) < EPS)
    return abs(p - l.b);
  return fabsl(cross(d, p - l.a)) / abs(d);
}
ld distanceBetweenSegments(const Line& a,
  const Line& b) { //2 線分間の距離
  if (isIntersect(a, b))
    return 0;
  return min(
    {distanceBetweenSegmentAndPoint(a, b.a),
      distanceBetweenSegmentAndPoint(
        a, b.b),
      distanceBetweenSegmentAndPoint(
        b, a.a),
      distanceBetweenSegmentAndPoint(
        b, a.b)});
}

ld SignedPolygonArea(const vector<Point>&
    p) { //多角形の符号付き面積
  ld s = 0;
  for (int i = 0, n = p.size(); i < n; i++)
    s += cross(p[i], p[(i + 1) % n]);
  return s / 2;
}
ld PolygonArea(
  const vector<Point>& p) { //多角形の面積
  return fabsl(SignedPolygonArea(p));
}

bool isConvex(
  const vector<Point>& p) { //多角形が凸か
  if (p.size() < 3)
    return false;
  int s = 0;
  for (int i = 0, n = p.size(); i < n;
    i++) {
    int t = sgn(cross(p[(i + 1) % n] - p[i],
      p[(i + 2) % n] - p[(i + 1) % n]));
    if (t && s && t != s)
      return false;
    if (t)
```

geometry.cpp (cont.)

```
      s = t;
  }
  return true;
}

int isContained(const vector<Point>& g,
  const Point&
    p) { //点 p が多角形 g の内/辺上/外か(2/1/0)
  bool in = 0;
  for (int i = 0, n = g.size(); i < n;
    i++) {
    Point a = g[i] - p,
          b = g[(i + 1) % n] - p;
    if (imag(a) > imag(b))
      swap(a, b);
    if (imag(a) <= EPS && EPS < imag(b) &&
      cross(a, b) < -EPS)
      in ^= 1;
    if (equal(cross(a, b), 0) &&
      dot(a, b) <= EPS)
      return 1;
  }
  return in ? 2 : 0;
}

vector<Point> ConvexHull(vector<Point> p,
  bool col =
    false) { //凸包を反時計回りに返す(
  ↪ col=true なら辺上も含む)
  sort(p.begin(), p.end(),
    [](Point a, Point b) {
      return real(a) != real(b)
        ? real(a) < real(b)
        : imag(a) < imag(b);
    });
  p.erase(unique(p.begin(), p.end(),
            [](Point a, Point b) {
              return abs(a - b) < EPS;
            }),
    p.end());

  int n = p.size(), k = 0;
  if (n <= 1)
    return p;
  vector<Point> h(2 * n);
  if (col) {
    bool all = 1;
    for (int i = 2; i < n; i++)
      if (fabsl(cross(p[1] - p[0],
            p[i] - p[0])) > EPS)
        all = 0;
    if (all)
      return p;
  }

  auto bad =
    [&](Point a, Point b,
      Point c) { //凸包に不要な中間点か
      ld x = cross(b - a, c - b);
      return col ? x < -EPS : x < EPS;
    };

  for (auto x : p) {
    while (
      k >= 2 && bad(h[k - 2], h[k - 1], x))
      --k;
```

geometry.cpp (cont.)

```
    h[k++] = x;
  }
  for (int i = n - 2, t = k + 1; i >= 0;
    i--) {
    while (k >= t &&
      bad(h[k - 2], h[k - 1], p[i]))
      --k;
    h[k++] = p[i];
  }
  h.resize(k - 1);
  return h;
}

int isIntersect(const Circle& a,
  const Circle&
    b) { //2 円の位置関係(返り値=共通接線数,
  ↪ -1 は同一円)
  ld d = abs(a.p - b.p), s = a.r + b.r,
     t = fabsl(a.r - b.r);
  if (equal(d, 0) && equal(a.r, b.r))
    return -1;
  if (d > s + EPS)
    return 4; //離れている
  if (equal(d, s))
    return 3; //外接
  if (d < t - EPS)
    return 0; //内包
  if (equal(d, t))
    return 1; //内接
  return 2; //2 点で交わる
}

Circle inCircle(const Point& a,
  const Point& b,
  const Point& c) { //三角形 abc の内接円
  ld A = abs(b - c), B = abs(a - c),
     C = abs(a - b);
  assert(A + B + C > EPS &&
    fabsl(cross(b - a, c - a)) > EPS);
  Point p =
    (a * A + b * B + c * C) / (A + B + C);
  return {p,
    distanceBetweenLineAndPoint({a, b}, p)};
}

vector<Point> crossPoint(const Circle& c,
  const Line& l) { //円 c と直線 l の交点
  ld d =
    distanceBetweenLineAndPoint(l, c.p);
  if (d > c.r + EPS)
    return {};
  Point h = projection(l, c.p);
  if (equal(d, c.r))
    return {h};
  Point u = unitVector(l.b - l.a) *
    sqrt(max<ld>(0, c.r * c.r - d * d));
  return {h - u, h + u};
}

vector<Point> crossPoint(const Circle& a,
  const Circle& b) { //2 円の交点
  int m = isIntersect(a, b);
  if (m == 4 || m == 0 || m == -1)
    return {};

  ld d = abs(b.p - a.p);
  ld x = (a.r * a.r + d * d - b.r * b.r) /
```

geometry.cpp (cont.)

```
    (2 * d);
  ld y =
    sqrt(max<ld>(0, a.r * a.r - x * x));

  Point u = (b.p - a.p) / d,
        h = a.p + u * x;
  if (m == 1 || m == 3)
    return {h};

  Point v = u * Point(0, y);
  return {h + v, h - v};
}

vector<Point> tangentToCircle(
  const Point& p,
  const Circle& c) { //点 p から円 c への接点
  ld d = norm(c.p - p);
  if (d < c.r * c.r - EPS)
    return {};
  return crossPoint(c,
    Circle(
      p, sqrt(max<ld>(0, d - c.r * c.r))));
}

vector<Line> tangent(const Circle& a,
  const Circle& b) { //2 円の共通接線
  vector<Line> r;
  ld g = abs(a.p - b.p);
  if (equal(g, 0))
    return r;

  Point u = unitVector(b.p - a.p),
        v = normalVector(u);

  for (int s : {-1, 1}) {
    ld h = (a.r + b.r * s) / g,
       q = 1 - h * h;
    if (q < -EPS)
      continue;

    if (fabsl(q) < EPS) {
      Point w = h > 0 ? u : -u;
      r.push_back(
        {a.p + w * a.r, a.p + w * a.r + v});
    } else {
      Point U = u * h,
            V = v * sqrt(max<ld>(0, q));
      for (int t : {1, -1}) {
        Point w = U + V * (ld)t;
        r.push_back({a.p + w * a.r,
          b.p - w * (b.r * s)});
      }
    }
  }
  return r;
}
```

hld.cpp

```
template <typename Graph>
struct HeavyLightDecomposition {
  //Edge query:: seg.set(hld.get_edge(u[i],
  v[i]),w[i]);
  //auto f=[&](ll u,ll v){ans+=seg.prod(u,
  v);return;};
  //hld.path_query<function<void(ll,ll)>>(
  uc,vc,false,f);
private:
```

hld.cpp (cont.)

```
  void dfs_sz(int now, int p) {
    // 1st dfs
    par[now] = p;
    depth[now] =
      (p == -1 ? 0 : depth[p] + 1);
    size[now] = 1;

    if (G[now].size() && G[now][0] == p)
      swap(G[now][0], G[now].back());
    for (auto& e : G[now]) {
      if (e == p)
        continue;
      dfs_sz(e, now);
      size[now] += size[e];
      if (size[e] > size[G[now][0]]) {
        swap(G[now][0], e);
      }
    }
  }
  void dfs_hld(int now, int p) {
    id[now] = id_k;
    vertex[id_k++] = now;
    for (auto& e : G[now]) {
      if (e == p)
        continue;
      bool heavy = (e == G[now][0]);
      head[e] = (heavy ? head[now] : e);
      dfs_hld(e, now);
    }
  }
  vector<pair<int, int>> ascend(
    int u, int v) const {
    vector<pair<int, int>> res;
    while (head[u] != head[v]) {
      res.push_back({id[u], id[head[u]]});
      u = par[head[u]];
    }
    if (u != v)
      res.push_back({id[u], id[v] + 1});
    return res;
  }
  vector<pair<int, int>> descend(
    int u, int v) const {
    if (u == v)
      return {};
    if (head[u] == head[v])
      return {{id[u] + 1, id[v]}};
    auto res = descend(u, par[head[v]]);
    res.push_back({id[head[v]], id[v]});
    return res;
  }

public:
  Graph& G;
  int root, id_k;
  vector<int> size, par, depth, head, id,
    vertex;
  //head[v]:=v を通る heavy path のうち最も根側
  にある頂点
  //id[v]:=v を行きがけ順で何番目に通るか
  //vertex[i]:=行きがけ順で i 番目に通る頂点番
  号
  HeavyLightDecomposition(
    Graph& _g, int _root = 0)
      : G(_g), root(_root), id_k(0),
        size(G.size(), 0),
        par(G.size(), root),
```

hld.cpp (cont.)

```
        depth(G.size(), 0),
        head(G.size(), 0), id(G.size(), 0),
        vertex(G.size(), 0) {
    dfs_sz(root, -1);
    dfs_hld(root, -1);
  }

  template <typename F>
  void path_query(
    int u, int v, bool vertex, const F& f) {
    int l = lca(u, v);
    for (auto [l, r] : ascend(u, l)) {
      int s = l + 1, t = r;
      s > t ? f(t, s) : f(s, t);
    }
    if (vertex)
      f(id[l], id[l] + 1);
    for (auto [l, r] : descend(l, v)) {
      int s = l, t = r + 1;
      s > t ? f(t, s) : f(s, t);
    }
  }

  template <typename F>
  void path_noncommutative_query(
    int u, int v, bool vertex, const F& f) {
    int l = lca(u, v);
    for (auto [l, r] : ascend(u, l))
      f(l + 1, r);
    if (vertex)
      f(id[l], id[l] + 1);
    for (auto [l, r] : descend(l, v))
      f(l, r + 1);
  }

  int level_ancestor(int v, int d) {
    //v の祖先で深さ d にあるものは?
    assert(depth[v] >= d);
    while (depth[head[v]] > d) {
      v = par[head[v]];
    }
    return vertex[id[v] - (depth[v] - d)];
  }

  int lca(int u, int v) {
    while (head[u] != head[v]) {
      if (id[u] > id[v])
        swap(u, v);
      v = par[head[v]];
    }
    return (id[u] < id[v] ? u : v);
  }

  int dist(int u, int v) {
    return depth[u] + depth[v] -
      2 * depth[lca(u, v)];
  }
  int get_edge(int u, int v) {
    if (abs(depth[u] - depth[v]) != 1)
      return -1;
    if (depth[u] > depth[v])
      return id[u];
    return id[v];
  }
};
```

hld_short.cpp

```
#include <bits/stdc++.h>
using namespace std;
vector<vector<int>> g;
vector<int> sz, in, out, nxt;
int t;
void dfs_sz(int v = 0) {
  sz[v] = 1;
  for (auto& u : g[v]) {
    dfs_sz(u);
    sz[v] += sz[u];
    if (sz[u] > sz[g[v][0]]) {
      swap(u, g[v][0]);
    }
  }
}
void dfs_hld(int v = 0) {
  in[v] = t++;
  for (auto u : g[v]) {
    nxt[u] = (u == g[v][0] ? nxt[v] : u);
    dfs_hld(u);
  }
  out[v] = t;
}
```

lazy_segtree.cpp

```
template <class S, S op(S, S), S e(),
  class F, S mapping(F, S),
  F composition(F, F), F id()>
struct lazy_segtree {
  int n, sz;
  vector<S> d;
  vector<F> lz;
  lazy_segtree(int m = 0) : n(1), sz(m) {
    while (n < m)
      n *= 2;
    d.assign(2 * n, e());
    lz.assign(2 * n, id());
  }
  lazy_segtree(vector<S> a)
      : lazy_segtree(a.size()) {
    copy(a.begin(), a.end(), d.begin() + n);
    for (int i = n - 1; i; i--)
      d[i] = op(d[2 * i], d[2 * i + 1]);
  }
  void all(int k, F f) {
    d[k] = mapping(f, d[k]);
    lz[k] = composition(f, lz[k]);
  }
  void push(int k) {
    all(2 * k, lz[k]);
    all(2 * k + 1, lz[k]);
    lz[k] = id();
  }
  void set(int p, S x) {
    set(p, x, 1, 0, n);
  }
  void set(
    int p, S x, int k, int l, int r) {
    if (r - l == 1) {
      d[k] = x;
      lz[k] = id();
      return;
    }
    push(k);
    int m = (l + r) / 2;
    p < m ? set(p, x, 2 * k, l, m)
          : set(p, x, 2 * k + 1, m, r);
```

lazy_segtree.cpp (cont.)

```
    d[k] = op(d[2 * k], d[2 * k + 1]);
  }
  S get(int p) { return prod(p, p + 1); }
  S prod(int a, int b) {
    return prod(a, b, 1, 0, n);
  }
  S prod(
    int a, int b, int k, int l, int r) {
    if (r <= a || b <= l)
      return e();
    if (a <= l && r <= b)
      return d[k];
    push(k);
    int m = (l + r) / 2;
    return op(prod(a, b, 2 * k, l, m),
      prod(a, b, 2 * k + 1, m, r));
  }
  S all_prod() { return d[1]; }
  void apply(int p, F f) {
    apply(p, p + 1, f);
  }
  void apply(int a, int b, F f) {
    apply(a, b, f, 1, 0, n);
  }
  void apply(int a, int b, F f, int k,
    int l, int r) {
    if (r <= a || b <= l)
      return;
    if (a <= l && r <= b) {
      all(k, f);
      return;
    }
    push(k);
    int m = (l + r) / 2;
    apply(a, b, f, 2 * k, l, m);
    apply(a, b, f, 2 * k + 1, m, r);
    d[k] = op(d[2 * k], d[2 * k + 1]);
  }

  template <class G>
  int max_right(int l, G g) {
    assert(g(e()));
    S x = e();
    int r = mr(l, g, x, 1, 0, n);
    return r < 0 ? sz : r;
  }
  template <class G>
  int mr(int a, G& g, S& x, int k, int l,
    int r) {
    if (r <= a || sz <= l)
      return -1;
    if (a <= l && r <= sz) {
      S y = op(x, d[k]);
      if (g(y)) {
        x = y;
        return -1;
      }
      if (r - l == 1)
        return l;
    }
    push(k);
    int m = (l + r) / 2;
    int q = mr(a, g, x, 2 * k, l, m);
    return q < 0
      ? mr(a, g, x, 2 * k + 1, m, r)
      : q;
  }
```

lazy_segtree.cpp (cont.)

```
  template <class G>
  int min_left(int r, G g) {
    assert(g(e()));
    S x = e();
    int l = ml(r, g, x, 1, 0, n);
    return l < 0 ? 0 : l;
  }
  template <class G>
  int ml(int b, G& g, S& x, int k, int l,
    int r) {
    if (b <= l)
      return -1;
    if (r <= b) {
      S y = op(d[k], x);
      if (g(y)) {
        x = y;
        return -1;
      }
      if (r - l == 1)
        return r;
    }
    push(k);
    int m = (l + r) / 2;
    int q = ml(b, g, x, 2 * k + 1, m, r);
    return q < 0 ? ml(b, g, x, 2 * k, l, m)
                 : q;
  }
};
```

lcp_array.cpp

```
template <class T>
vector<int> lcp_array(
  const T& s, const vector<int>& sa) {
  int n = s.size(), h = 0;
  vector<int> r(n), l(n - 1);
  for (int i = 0; i < n; i++)
    r[sa[i]] = i;
  for (int i = 0; i < n; i++)
    if (r[i]) {
      int j = sa[r[i] - 1];
      while (max(i, j) + h < n &&
        s[i + h] == s[j + h])
        h++;
      l[r[i] - 1] = h;
      if (h)
        h--;
    }
  return l;
}
```

linear_equation.cpp

```
// 行列 a をガウス消去する
// 返り値 : {rank, det}
//   rank : pivot_end 列目までを係数部分としたときの rank
//   det  : 消去中の pivot の積×行交換の符号
//          正方行列 a に対して全列を pivot 対象にした場合は det(a)
// diagonalize=true なら簡約行階段形にする
template <class T>
pair<int, T> GaussElimination(
  vector<vector<T>>& a, int pe = -1,
  bool diag = false) {
  if (a.empty())
    return {0, T(1)};
  int h = a.size(), w = a[0].size(), r = 0;
  if (pe < 0)
    pe = w;
```

linear_equation.cpp (cont.)

```cpp
  T det = 1;
  for (int j = 0; j < pe; j++) {
    int p = r;
    while (p < h && a[p][j] == T(0))
      p++;
    if (p == h) {
      det = 0;
      continue;
    }
    if (p != r)
      swap(a[p], a[r]), det = -det;
    det *= a[r][j];
    if (diag) {
      T x = T(1) / a[r][j];
      for (int k = j; k < w; k++)
        a[r][k] *= x;
    }
    for (int i = diag ? 0 : r + 1; i < h;
      i++)
      if (i != r && a[i][j] != T(0)) {
        T x = a[i][j] / a[r][j];
        for (int k = j; k < w; k++)
          a[i][k] -= a[r][k] * x;
      }
    r++;
  }
  return {r, det};
}
// 線形方程式 Ax=b を解く
// 返り値 : 解なしなら {}
//          解ありなら {v,w1,...,wk}
//          ただし全ての解は
//              x = v + c1*w1 + ... + ck*wk
//          と表される
//   v       : 特殊解の 1 つ
//   w1..wk  : Ker(A)の基底 (自由変数に対応 )
//   k       : W-rank(A) = 解空間の次元
template <class T>
vector<vector<T>> LinearEquation(
  vector<vector<T>> a, vector<T> b) {
  int h = a.size(), w = a[0].size();
  for (int i = 0; i < h; i++)
    a[i].push_back(b[i]);
  int r =
    GaussElimination(a, w, true).first;
  for (int i = r; i < h; i++)
    if (a[i][w] != T(0))
      return {};

  vector<vector<T>> res(1, vector<T>(w));
  vector<int> p(w, -1);
  for (int i = 0, j = 0; i < r; i++) {
    while (a[i][j] == T(0))
      j++;
    p[j] = i;
    res[0][j] = a[i][w];
  }
  for (int j = 0; j < w; j++)
    if (p[j] < 0) {
      vector<T> x(w);
      x[j] = 1;
      for (int k = 0; k < j; k++)
        if (p[k] >= 0)
          x[k] = -a[p[k]][j];
      res.push_back(x);
    }
```

linear_equation.cpp (cont.)

```cpp
  return res;
}
```

linear_equation_bitset.cpp

```cpp
// F_2 上の線形方程式 Ax=b を解く
// a[i][0..W-1] : A の i 行目, a[i][W] : b[i]
// 返り値 : 解なしなら {}
//          解ありなら {v,w1,...,wk}
//          全解は x=v+c1*w1+...+ck*wk (ci in F_2)
//   v      : 特殊解
//   w1..wk : Ker(A)の基底 O(HW^2/64)
template <int MAX>
vector<bitset<MAX>> LinearEquation(
  vector<bitset<MAX + 1>> a, int W) {
  int H = a.size(), r = 0;
  vector<int> p(W, -1);
  for (int j = 0; j < W; j++) {
    int q = r;
    while (q < H && !a[q][j])
      q++;
    if (q == H)
      continue;
    swap(a[r], a[q]);
    p[j] = r;
    for (int i = 0; i < H; i++)
      if (i != r && a[i][j])
        a[i] ^= a[r];
    r++;
  }
  for (int i = r; i < H; i++)
    if (a[i][W])
      return {};

  vector<bitset<MAX>> res(1);
  for (int j = 0; j < W; j++)
    if (p[j] >= 0)
      res[0][j] = a[p[j]][W];

  for (int j = 0; j < W; j++)
    if (p[j] < 0) {
      bitset<MAX> x;
      x[j] = 1;
      for (int k = 0; k < W; k++)
        if (p[k] >= 0 && a[p[k]][j])
          x[k] = 1;
      res.push_back(x);
    }
  return res;
}
```

manacher.cpp

```cpp
template <class T>
vector<int> manacher(const T& s) {
  int n = s.size();
  vector<int> R(n);
  int i = 0, j = 0;
  while (i < s.size()) {
    while (i - j >= 0 && i + j < s.size() &&
      s[i - j] == s[i + j])
      ++j;
    R[i] = j;
    int k = 1;
    while (i - k >= 0 && k + R[i - k] < j)
      R[i + k] = R[i - k], ++k;
    i += k;
    j -= k;
  }
```

manacher.cpp (cont.)

```
  return R;
}
```

math.cpp

```
using ll = long long;
using ull = unsigned long long;

ll smod(ll x, ll m) {
  return (x %= m) < 0 ? x + m : x;
}

pair<ll, ll> igcd(ll a, ll b) {
  a = smod(a, b);
  if (!a)
    return {b, 0};
  ll s = b, t = a, x = 0, y = 1;
  while (t) {
    ll q = s / t;
    s -= t * q;
    x -= y * q;
    swap(s, t);
    swap(x, y);
  }
  if (x < 0)
    x += b / s;
  return {s, x};
}

ll pow_mod(ll x, ll n, int m) {
  ll r = 1 % m;
  x = smod(x, m);
  for (; n; n >>= 1, x = x * x % m)
    if (n & 1)
      r = r * x % m;
  return r;
}

ll inv_mod(ll x, ll m) {
  return igcd(x, m).second;
}

pair<ll, ll> crt(const vector<ll>& r,
  const vector<ll>& m) {
  ll R = 0, M = 1;
  for (int i = 0; i < (int)r.size(); i++) {
    ll x = smod(r[i], m[i]);
    auto [g, im] = igcd(M, m[i]);
    ll u = m[i] / g;
    if ((x - R) % g)
      return {0, 0};
    ll t = (__int128)smod((x - R) / g, u) *
      im % u;
    R += M * t;
    M *= u;
    R = smod(R, M);
  }
  return {R, M};
}

ll floor_sum(ll n, ll m, ll a, ll b) {
  ull z = 0;
  if (a < 0) {
    ull x = smod(a, m);
    z -= 1ULL * n * (n - 1) / 2 *
      ((x - a) / m);
    a = x;
  }
```

math.cpp (cont.)

```
  if (b < 0) {
    ull x = smod(b, m);
    z -= 1ULL * n * ((x - b) / m);
    b = x;
  }
  for (;;) {
    if (a >= m)
      z += 1ULL * n * (n - 1) / 2 * (a / m),
        a %= m;
    if (b >= m)
      z += 1ULL * n * (b / m), b %= m;
    ull y = (ull)a * n + b;
    if (y < (ull)m)
      return (ll)z;
    n = y / m;
    b = y % m;
    swap(a, m);
  }
}
```

maxflow.cpp

```
template <class Cap>
struct mf_graph {
  struct E {
    int to, rev;
    Cap cap;
  };
  struct edge {
    int from, to;
    Cap cap, flow;
  };
  int N;
  vector<vector<E>> g;
  vector<pair<int, int>> pos;

  mf_graph(int n = 0) : N(n), g(n) {}

  int add_edge(int s, int t, Cap c) {
    int a = g[s].size(),
        b = g[t].size() + (s == t);
    pos.push_back({s, a});
    g[s].push_back({t, b, c});
    g[t].push_back({s, a, 0});
    return (int)pos.size() - 1;
  }

  edge get_edge(int i) {
    auto [v, j] = pos[i];
    auto &e = g[v][j], &r = g[e.to][e.rev];
    return {v, e.to, e.cap + r.cap, r.cap};
  }

  vector<edge> edges() {
    vector<edge> r;
    for (int i = 0; i < (int)pos.size();
      i++)
      r.push_back(get_edge(i));
    return r;
  }

  void change_edge(int i, Cap c, Cap f) {
    auto [v, j] = pos[i];
    auto &e = g[v][j], &r = g[e.to][e.rev];
    e.cap = c - f;
    r.cap = f;
  }

```

maxflow.cpp (cont.)

```
  Cap flow(int s, int t) {
    return flow(
      s, t, numeric_limits<Cap>::max());
  }

  Cap flow(int s, int t, Cap lim) {
    Cap ans = 0;
    vector<int> lv(N), it(N);
    while (ans < lim) {
      fill(lv.begin(), lv.end(), -1);
      queue<int> q;
      q.push(s);
      lv[s] = 0;
      while (q.size()) {
        int v = q.front();
        q.pop();
        for (auto& e : g[v])
          if (e.cap && lv[e.to] < 0)
            lv[e.to] = lv[v] + 1,
            q.push(e.to);
      }
      if (lv[t] < 0)
        break;
      fill(it.begin(), it.end(), 0);

      auto dfs = [&](auto&& self, int v,
                     Cap f) -> Cap {
        if (v == t)
          return f;
        for (int& i = it[v];
          i < (int)g[v].size(); i++) {
          E& e = g[v][i];
          if (e.cap &&
            lv[e.to] == lv[v] + 1)
            if (Cap x = self(self, e.to,
                  min(f, e.cap))) {
              e.cap -= x;
              g[e.to][e.rev].cap += x;
              return x;
            }
        }
        return 0;
      };

      while (ans < lim) {
        Cap x = dfs(dfs, s, lim - ans);
        if (!x)
          break;
        ans += x;
      }
    }
    return ans;
  }

  vector<bool> min_cut(int s) {
    vector<bool> z(N);
    queue<int> q;
    q.push(s);
    z[s] = 1;
    while (q.size()) {
      int v = q.front();
      q.pop();
      for (auto& e : g[v])
        if (e.cap && !z[e.to])
          z[e.to] = 1, q.push(e.to);
    }
```

maxflow.cpp (cont.)

```
    return z;
  }
};
```

mincostflow.cpp

```
template <class Cap, class Cost>
struct mcf_graph {
  struct E {
    int to, rev;
    Cap cap;
    Cost cost;
  };
  struct edge {
    int from, to;
    Cap cap, flow;
    Cost cost;
  };
  int N;
  vector<vector<E>> g;
  vector<pair<int, int>> pos;

  mcf_graph(int n = 0) : N(n), g(n) {}

  int add_edge(
    int s, int t, Cap c, Cost w) {
    int a = g[s].size(),
        b = g[t].size() + (s == t);
    pos.push_back({s, a});
    g[s].push_back({t, b, c, w});
    g[t].push_back({s, a, 0, -w});
    return (int)pos.size() - 1;
  }

  edge get_edge(int i) {
    auto [v, j] = pos[i];
    auto &e = g[v][j], &r = g[e.to][e.rev];
    return {v, e.to, e.cap + r.cap, r.cap,
      e.cost};
  }

  vector<edge> edges() {
    vector<edge> r;
    for (int i = 0; i < (int)pos.size();
      i++)
      r.push_back(get_edge(i));
    return r;
  }

  pair<Cap, Cost> flow(int s, int t) {
    return flow(
      s, t, numeric_limits<Cap>::max());
  }

  pair<Cap, Cost> flow(
    int s, int t, Cap lim) {
    return slope(s, t, lim).back();
  }

  vector<pair<Cap, Cost>> slope(
    int s, int t) {
    return slope(
      s, t, numeric_limits<Cap>::max());
  }

  vector<pair<Cap, Cost>> slope(
```

mincostflow.cpp (cont.)

```
    int s, int t, Cap lim) {
    const Cost INF =
      numeric_limits<Cost>::max();
    vector<Cost> h(N), d(N);
    vector<int> pv(N), pe(N);

    Cap f = 0;
    Cost c = 0, last = -1;
    vector<pair<Cap, Cost>> res{{0, 0}};

    while (f < lim) {
      fill(d.begin(), d.end(), INF);
      d[s] = 0;

      priority_queue<pair<Cost, int>,
        vector<pair<Cost, int>>,
        greater<pair<Cost, int>>>
        q;
      q.push({0, s});

      while (q.size()) {
        auto [dv, v] = q.top();
        q.pop();
        if (dv != d[v])
          continue;

        for (int i = 0;
          i < (int)g[v].size(); i++) {
          E& e = g[v][i];
          if (!e.cap)
            continue;

          __int128 nd = (__int128)dv +
            e.cost + h[v] - h[e.to];
          if (nd < d[e.to]) {
            d[e.to] = (Cost)nd;
            pv[e.to] = v;
            pe[e.to] = i;
            q.push({(Cost)nd, e.to});
          }
        }
      }

      if (d[t] == INF)
        break;

      for (int v = 0; v < N; v++)
        if (d[v] < INF)
          h[v] += d[v];

      Cap x = lim - f;
      for (int v = t; v != s; v = pv[v])
        x = min(x, g[pv[v]][pe[v]].cap);

      for (int v = t; v != s; v = pv[v]) {
        E& e = g[pv[v]][pe[v]];
        e.cap -= x;
        g[v][e.rev].cap += x;
      }

      Cost w = h[t] - h[s];
      f += x;
      c += x * w;

      if (last == w)
        res.pop_back();
      res.push_back({f, c});
```

mincostflow.cpp (cont.)

```
        last = w;
      }

      return res;
    }
};
```

mo.cpp

```
struct Mo {
  int n, l = 0, r = 0; // 現在の区間[l,r)
  vector<pair<int, int>>
    lr; //[l,r) add や erase では更新後の l,
  r を参照できる
  Mo(int N) : n(N) {} //i 番目の答えが入る
  void add(
    int l, int r) { // [l,r) のクエリを追加
    lr.emplace_back(l, r);
  }
  template <class AL, class AR, class EL,
    class ER, class O>
  void build(const AL& al, const AR& ar,
    const EL& el, const ER& er,
    const O& out) {
    int Q = lr.size();
    if (!Q)
      return;
    int B = max(1, n / (int)sqrt(Q));
    vector<int> o(Q);
    iota(o.begin(), o.end(), 0);
    sort(o.begin(), o.end(),
      [&](int a, int b) {
        int x = lr[a].first / B,
            y = lr[b].first / B;
        return x != y
          ? x < y
          : (x & 1 ? lr[a].second >
                lr[b].second
                 : lr[a].second <
                lr[b].second);
      });
    l = r = 0;
    for (int i : o) {
      auto [L, R] = lr[i];
      while (l > L)
        al(--l);
      while (r < R)
        ar(r++);
      while (l < L)
        el(l++);
      while (r > R)
        er(--r);
      out(i);
    }
  }
  template <class A, class E, class O>
  void build(const A& add, const E& erase,
    const O& out) { //add(i):
    build(add, add, erase, erase, out);
  }
};
// auto add_left = [&](int i){};
  左側の i 番目の要素を加える
// auto erase = [&](int i){};
// auto out = [&](int q){}; 現在のクエリ(
  q 番目)の答えを求める
```

modint_convolution.cpp

```cpp
struct mint {
  static const int M = 998244353;
  int x;
  mint(long long v = 0)
      : x((v % M + M) % M) {}
  int val() const { return x; }
  static int mod() { return M; }
  mint& operator+=(mint a) {
    if ((x += a.x) >= M)
      x -= M;
    return *this;
  }
  mint& operator-=(mint a) {
    if ((x -= a.x) < 0)
      x += M;
    return *this;
  }
  mint& operator*=(mint a) {
    x = (long long)x * a.x % M;
    return *this;
  }
  mint pow(long long n) const {
    mint a = *this, r = 1;
    for (; n; n >>= 1, a *= a)
      if (n & 1)
        r *= a;
    return r;
  }
  mint inv() const { return pow(M - 2); }
  mint& operator/=(mint a) {
    return *this *= a.inv();
  }
  friend mint operator+(mint a, mint b) {
    return a += b;
  }
  friend mint operator-(mint a, mint b) {
    return a -= b;
  }
  friend mint operator*(mint a, mint b) {
    return a *= b;
  }
  friend mint operator/(mint a, mint b) {
    return a /= b;
  }
};
using modint998244353 = mint;
void ntt(vector<mint>& a, bool inv = 0) {
  int n = a.size();
  for (int i = 1, j = 0; i < n; i++) {
    int b = n >> 1;
    for (; j & b; b >>= 1)
      j ^= b;
    j ^= b;
    if (i < j)
      swap(a[i], a[j]);
  }
  for (int l = 2; l <= n; l <<= 1) {
    mint w = mint(3).pow((mint::M - 1) / l);
    if (inv)
      w = w.inv();
    for (int i = 0; i < n; i += l) {
      mint z = 1;
      for (int j = 0; j < l / 2; j++) {
        mint x = a[i + j],
             y = a[i + j + l / 2] * z;
        a[i + j] = x + y;
        a[i + j + l / 2] = x - y;
        z *= w;
      }
    }
  }
  if (inv) {
    mint z = mint(n).inv();
    for (auto& x : a)
      x *= z;
  }
}
vector<mint> convolution(
  vector<mint> a, vector<mint> b) {
  if (a.empty() || b.empty())
    return {};
  int n = a.size() + b.size() - 1, z = 1;
  while (z < n)
    z <<= 1;
  a.resize(z);
  b.resize(z);
  ntt(a);
  ntt(b);
  for (int i = 0; i < z; i++)
    a[i] *= b[i];
  ntt(a, 1);
  a.resize(n);
  return a;
}
```

rerooting.cpp

```cpp
/*
    - n 頂点の木 T が与えられている

    - RerootingDP<E,V,merge,e,put_edge,put_vertex> g(n) : コンストラクタ O(n)
        E : 親の dp の計算をする際に用いる値の型
        V : 頂点の dp の値の型
        E put_edge (V v, int i) : 頂点 v を根とする部分木の dp の値に辺 i を追加した時に得られる値
        E merge(E e1, E e2) :  2 つの子について求めた put_edge の値を結合する
            merge は可換モノイドによる結合を要求される
        E e() : merge の単位元を返す関数
        V put_vertex(E e, int u) : 全ての子について merge した値から，頂点 u の dp[u]を計算する
    - void add_edge(int u, int v, int idx, int xdi) : 頂点 u->v の辺番号 idx の辺と
        頂点 v->u の辺番号 xdi の辺を追加する (ちょうど n-1 回呼ぶ．ならし O(1))
    - V bulid(int v): 頂点 v を根とした時の，dp[v]の値を返す (1 回までしか呼べない O(n))
    - vector<V> reroot(): 長さ n の配列( i 番目には，頂点 i を根とした時の，dp[i]の値)を返す
        (先に build をちょうど 1 回呼んでいる必要あり O(n))
*/

template <class E, class V,
  E (*merge)(E, E), E (*e)(),
  E (*put_edge)(V, int),
  V (*put_vertex)(E, int)>
```

rerooting.cpp (cont.)

```cpp
struct RerootingDP {
  struct edge {
    int to, idx, xdi;
  };
  RerootingDP(int n_ = 0)
      : n(n_), inner_edge_id(0) {
    es.resize(2 * n - 2);
    start.resize(2 * n - 2);
    if (n == 1)
      es_build();
  }
  void add_edge(
    int u, int v, int idx, int xdi) {
    start[inner_edge_id] = u;
    es[inner_edge_id] = {v, idx, xdi};
    inner_edge_id++;
    start[inner_edge_id] = v;
    es[inner_edge_id] = {u, xdi, idx};
    inner_edge_id++;
    if (inner_edge_id == 2 * n - 2) {
      es_build();
    }
  }
  vector<V> build(int root_ = 0) {
    root = root_;
    vector<V> subdp(n);
    subdp[0] = put_vertex(e(), 0);
    outs.resize(n);
    vector<int> geta(n + 1, 0);
    for (int i = 0; i < n; i++)
      geta[i + 1] =
        start[i + 1] - start[i] - 1;
    geta[root + 1]++;
    for (int i = 0; i < n; i++)
      geta[i + 1] += geta[i];
    auto dfs = [&](auto sfs, int v,
                   int f) -> void {
      E val = e();
      for (int i = start[v];
        i < start[v + 1]; i++) {
        if (es[i].to == f) {
          swap(es[start[v + 1] - 1], es[i]);
        }
        if (es[i].to == f)
          continue;
        sfs(sfs, es[i].to, v);
        E nval = put_edge(
          subdp[es[i].to], es[i].idx);
        outs[geta[v]++] = nval;
        val = merge(val, nval);
      }
      subdp[v] = put_vertex(val, v);
    };
    dfs(dfs, root, -1);
    return subdp;
  }
  vector<V> reroot() {
    vector<E> reverse_edge(n);
    reverse_edge[root] = e();
    vector<V> answers(n);
    auto dfs = [&](
                  auto sfs, int v) -> void {
      int le = outs_start(v);
      int ri = outs_start(v + 1);
      int siz = ri - le;
      vector<E> rui(siz + 1);
      rui[siz] = e();
      for (int i = siz - 1; i >= 0; i--) {
        rui[i] =
          merge(outs[le + i], rui[i + 1]);
      }
      answers[v] = put_vertex(
        merge(rui[0], reverse_edge[v]), v);
      E lui = e();
      for (int i = 0; i < siz; i++) {
        V rdp = put_vertex(
          merge(merge(lui, rui[i + 1]),
            reverse_edge[v]),
          v);
        reverse_edge[es[start[v] + i].to] =
          put_edge(
            rdp, es[start[v] + i].xdi);
        lui = merge(lui, outs[le + i]);
        sfs(sfs, es[start[v] + i].to);
      }
    };
    dfs(dfs, root);
    return answers;
  }

private:
  int n, root, inner_edge_id;
  vector<E> outs;
  vector<edge> es;
  vector<int> start;
  int outs_start(int v) {
    int res = start[v] - v;
    if (root < v)
      res++;
    return res;
  }
  void es_build() {
    vector<edge> nes(2 * n - 2);
    vector<int> nstart(n + 2, 0);
    for (int i = 0; i < 2 * n - 2; i++)
      nstart[start[i] + 2]++;
    for (int i = 0; i < n; i++)
      nstart[i + 1] += nstart[i];
    for (int i = 0; i < 2 * n - 2; i++)
      nes[nstart[start[i] + 1]++] = es[i];
    swap(es, nes);
    swap(start, nstart);
  }
};
```

rollback_mo.cpp

```cpp
struct MoRollBack {
  int w, l = 0, r = 0; // 現在の区間 [l,r)
  vector<pair<int, int>> q;
  vector<int> ord;
  MoRollBack(int N, int Q)
      : w(max(1, (int)sqrt(max(1, N)))),
        ord(Q) {
    iota(ord.begin(), ord.end(), 0);
  }
  void add(
    int L, int R) { // クエリ[L,R)を追加
    q.push_back({L, R});
  }
  template <class A, class REM, class RS,
    class S, class B>
  void run(const A& add, const REM& rem,
    const RS& reset, const S& snapshot,
    const B& rollback) { // 全クエリを処理
```

rollback_mo.cpp (cont.)

```
    assert(q.size() == ord.size());
    sort(ord.begin(), ord.end(),
      [&](int i, int j) {
        int a = q[i].first / w,
            b = q[j].first / w;
        return a != b
          ? a < b
          : q[i].second < q[j].second;
      });
    l = r = 0;
    reset();
    for (int i : ord)
      if (q[i].second - q[i].first < w) {
        auto [L, R] = q[i];
        l = r = L;
        while (r < R) {
          int j = r++;
          add(
            j); // add 後の区間 [l,r)
  を参照できる
        }
        rem(
          i); // [l,r) はクエリ区間そのもの
        l = r = 0;
        rollback(); // rollback 後は空区間 [
  0,0)
      }
    int last = -1;
    for (int i : ord)
      if (q[i].second - q[i].first >= w) {
        auto [L, R] = q[i];
        int b = L / w, base = (b + 1) * w;
        if (b != last) {
          last = b;
          l = r = base;
          reset(); // 空区間 [base,base)
        }
        while (r < R) {
          int j = r++;
          add(j); // 右に追加後
        }
        snapshot();
        while (l > L)
          add(--l); // 左に追加後
        rem(i); // [l,r)=[L,R)
        l = base;
        rollback(); // [base,r) まで戻す
      }
  }
};
/* 区間内の異なる値の個数
int N,Q;
cin>>N>>Q;
vector<int>A(N);
for(int &x:A)cin>>x;
MoRollBack mo(N,Q);
for(int i=0;i<Q;i++){
    int l,r;
    cin>>l>>r;
    mo.add(l,r);
}
vector<int>cnt(1000001),ans(Q);
vector<int>hist;
int distinct=0,snap=0;
auto undo=[&](){
    int i=hist.back();
    hist.pop_back();
```

rollback_mo.cpp (cont.)

```

    if(--cnt[A[i]]==0)
        distinct--;
};
auto add=[&](int i){ // i を追加
    hist.push_back(i);

    if(cnt[A[i]]++==0)
        distinct++;

    cout<<"add "<<i<<" -> ["
        <<mo.l<<","<<mo.r<<")\n";
};
auto rem=[&](int q){ // q 番目の答えを記録
    ans[q]=distinct;

    cout<<"query "<<q<<" = ["
        <<mo.l<<","<<mo.r<<")\n";
};
auto reset=[&](){ // 状態を空にする
    while(!hist.empty())undo();
    snap=0;
};
auto snapshot=[&](){ // 現在位置を保存
    snap=hist.size();
};
auto rollback=[&](){ // snapshot まで巻き戻す
    while((int)hist.size()>snap)
        undo();
};
mo.run(add,rem,reset,snapshot,rollback);
for(int x:ans)
    cout<<x<<"\n";
*/
```

rolling_hash.cpp

```
struct rolling_hash {
  using ull = unsigned long long;
  using u128 = __uint128_t;
  static constexpr ull M = (1ULL << 61) - 1;
  static ull mul(ull a, ull b) {
    u128 x = (u128)a * b;
    ull y = (x >> 61) + (x & M);
    return y >= M ? y - M : y;
  }
  inline static ull B =
    chrono::steady_clock::now()
        .time_since_epoch()
        .count() %
      (M - 256) +
    256;
  vector<ull> h, p;
  rolling_hash(const string& s)
      : h(s.size() + 1),
        p(s.size() + 1, 1) {
    for (int i = 0; i < (int)s.size(); i++)
      p[i + 1] = mul(p[i], B),
            h[i + 1] =
              (mul(h[i], B) +
                (unsigned char)s[i] + 1) %
        M;
  }
  ull get(int l, int r) const {
    ull x = h[r] + M - mul(h[l], p[r - l]);
    return x >= M ? x - M : x;
  }
};
```

scc.cpp

```cpp
struct scc_graph {
  int n;
  vector<vector<int>> g, rg;
  scc_graph(int n) : n(n), g(n), rg(n) {}
  void add_edge(int a, int b) {
    g[a].push_back(b);
    rg[b].push_back(a);
  }
  vector<vector<int>> scc() {
    vector<int> vis(n), ord;
    for (int s = 0; s < n; s++)
      if (!vis[s]) {
        stack<pair<int, int>> q;
        q.push({s, 0});
        vis[s] = 1;
        while (q.size()) {
          auto& [v, i] = q.top();
          if (i < (int)g[v].size()) {
            int u = g[v][i++];
            if (!vis[u])
              vis[u] = 1, q.push({u, 0});
          } else
            ord.push_back(v), q.pop();
        }
      }
    fill(vis.begin(), vis.end(), 0);
    reverse(ord.begin(), ord.end());
    vector<vector<int>> res;
    for (int s : ord)
      if (!vis[s]) {
        res.push_back({});
        stack<int> q;
        q.push(s);
        vis[s] = 1;
        while (q.size()) {
          int v = q.top();
          q.pop();
          res.back().push_back(v);
          for (int u : rg[v])
            if (!vis[u])
              vis[u] = 1, q.push(u);
        }
      }
    return res;
  }
};
```

segtree.cpp

```cpp
template <class S, S op(S, S), S e()>
struct segtree {
  int n, sz;
  vector<S> d;
  segtree(int m = 0) : n(1), sz(m) {
    while (n < m)
      n *= 2;
    d.assign(2 * n, e());
  }
  segtree(vector<S> a) : segtree(a.size()) {
    copy(a.begin(), a.end(), d.begin() + n);
    for (int i = n - 1; i; i--)
      d[i] = op(d[2 * i], d[2 * i + 1]);
  }
  void set(int p, S x) {
    for (d[p += n] = x; p /= 2;)
      d[p] = op(d[2 * p], d[2 * p + 1]);
  }
  S get(int p) { return d[p + n]; }
```

segtree.cpp (cont.)

```cpp
  S prod(int l, int r) {
    S x = e(), y = e();
    for (l += n, r += n; l < r;
      l /= 2, r /= 2) {
      if (l & 1)
        x = op(x, d[l++]);
      if (r & 1)
        y = op(d[--r], y);
    }
    return op(x, y);
  }
  S all_prod() { return d[1]; }

  template <class G>
  int max_right(int l, G g) {
    assert(g(e()));
    S x = e();
    int r = mr(l, g, x, 1, 0, n);
    return r < 0 ? sz : r;
  }
  template <class G>
  int mr(int a, G& g, S& x, int k, int l,
    int r) {
    if (r <= a || sz <= l)
      return -1;
    if (a <= l && r <= sz) {
      S y = op(x, d[k]);
      if (g(y)) {
        x = y;
        return -1;
      }
      if (r - l == 1)
        return l;
    }
    int m = (l + r) / 2,
        q = mr(a, g, x, 2 * k, l, m);
    return q < 0
      ? mr(a, g, x, 2 * k + 1, m, r)
      : q;
  }
  template <class G>
  int min_left(int r, G g) {
    assert(g(e()));
    S x = e();
    int l = ml(r, g, x, 1, 0, n);
    return l < 0 ? 0 : l;
  }
  template <class G>
  int ml(int b, G& g, S& x, int k, int l,
    int r) {
    if (b <= l)
      return -1;
    if (r <= b) {
      S y = op(d[k], x);
      if (g(y)) {
        x = y;
        return -1;
      }
      if (r - l == 1)
        return r;
    }
    int m = (l + r) / 2,
        q = ml(b, g, x, 2 * k + 1, m, r);
    return q < 0 ? ml(b, g, x, 2 * k, l, m)
                 : q;
  }
};
```

suffix_array.cpp

```cpp
template <class T>
vector<int> suffix_array(vector<T> s) {
  int n = s.size();
  if (!n)
    return {};
  int N = n + 1;
  vector<int> p(N), c(N), q(N), nc(N),
    cnt(N);
  p[0] = n;
  iota(p.begin() + 1, p.end(), 0);
  sort(p.begin() + 1, p.end(),
    [&](int i, int j) {
      return s[i] < s[j];
    });
  for (int i = 1; i < N; i++)
    c[p[i]] = i == 1 ? 1
                     : c[p[i - 1]] +
        (s[p[i - 1]] != s[p[i]]);
  for (int k = 1; k < N; k *= 2) {
    for (int i = 0; i < N; i++)
      q[i] = (p[i] - k + N) % N;
    fill(cnt.begin(), cnt.end(), 0);
    for (int x : q)
      cnt[c[x]]++;
    for (int i = 1; i < N; i++)
      cnt[i] += cnt[i - 1];
    for (int i = N - 1; i >= 0; i--)
      p[--cnt[c[q[i]]]] = q[i];
    nc[p[0]] = 0;
    for (int i = 1; i < N; i++)
      nc[p[i]] = nc[p[i - 1]] +
        (c[p[i - 1]] != c[p[i]] ||
          c[(p[i - 1] + k) % N] !=
            c[(p[i] + k) % N]);
    c.swap(nc);
  }
  return {p.begin() + 1, p.end()};
}
vector<int> suffix_array(string s) {
  return suffix_array(vector<unsigned char>(
    s.begin(), s.end()));
}
vector<int> suffix_array(
  vector<int> s, int) {
  return suffix_array<int>(move(s));
}
```

template.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
#define rep(i, n)
#define all(a) begin(a), end(a)
using ll = long long;
using vi = vector<int>;
using vl = vector<ll>;
template <class T>
bool chmax(T& a, const T& b) {
  if (a < b) {
    a = b;
    return 1;
  }
  return 0;
}
template <class T>
bool chmin(T& a, const T& b) {
```

template.cpp (cont.)

```cpp
  if (b < a) {
    a = b;
    return 1;
  }
  return 0;
}

void solve() {
}

signed main() {
  cin.tie(0);
  ios::sync_with_stdio(0);
  cout << fixed << setprecision(20);
  int TT;
  TT = 1; //cin >> TT;
  solve();
}
```

trie.cpp

```cpp
template <int K, char B>
struct
  trie { //K 文字種,B を先頭文字とする文字列 Tr
  ie
  struct N {
    int nx[K], end = 0;
    N() { fill(nx, nx + K, -1); }
  };
  vector<N> t{N()};

  void insert(
    string s) { //文字列 s を追加する
    int v = 0;
    for (char c : s) {
      int x = c -
        B; //文字 c に対応する子番号
    連続でない文字を取りたいなら id 関数を作る
      if (t[v].nx[x] < 0)
        t[v].nx[x] = t.size(),
        t.emplace_back();
      v = t[v].nx[x];
    }
    t[v].end++;
  }

  int find(string
      s) { //文字列 s に対応する頂点番号を返す
    (存在しなければ-1)
    int v = 0;
    for (char c : s) {
      v = t[v].nx[c - B];
      if (v < 0)
        return -1;
    }
    return v;
  }
};
```

twosat.cpp

```cpp
struct two_sat {
  int n;
  vector<vector<int>> g, rg;
  vector<bool> a;
  two_sat(int N)
      : n(N), g(2 * N), rg(2 * N), a(N) {}

```

twosat.cpp (cont.)

```cpp
  void add(int x, int y) {
    g[x].push_back(y);
    rg[y].push_back(x);
  }
  void add_clause(
    int i, bool f, int j, bool h) {
    add(2 * i + !f, 2 * j + h);
    add(2 * j + !h, 2 * i + f);
  }

  bool satisfiable() {
    int N = 2 * n;
    vector<int> v(N), o, c(N);
    auto dfs = [&](auto&& self,
                   int x) -> void {
      v[x] = 1;
      for (int y : g[x])
        if (!v[y])
          self(self, y);
      o.push_back(x);
    };
    for (int i = 0; i < N; i++)
      if (!v[i])
        dfs(dfs, i);

    fill(v.begin(), v.end(), 0);
    reverse(o.begin(), o.end());
    int k = 0;
    auto rdfs = [&](auto&& self,
                    int x) -> void {
      v[x] = 1;
      c[x] = k;
      for (int y : rg[x])
        if (!v[y])
          self(self, y);
    };
    for (int x : o)
      if (!v[x])
        rdfs(rdfs, x), k++;

    for (int i = 0; i < n; i++) {
      if (c[2 * i] == c[2 * i + 1])
        return false;
      a[i] = c[2 * i] < c[2 * i + 1];
    }
    return true;
  }

  vector<bool> answer() { return a; }
};
```

wavelet_matrix.cpp

```cpp
template <class T,
  int LOG = numeric_limits<
    make_unsigned_t<T>>::
    digits> //LOGは log_2 A_i より大きい値を
  選択
struct
  WaveletMatrix { //WaveletMatrix<int,10>
  など
  using U = make_unsigned_t<T>;
  int n;
  array<int, LOG> mid{};
  array<vector<int>, LOG> s;
  WaveletMatrix(const vector<T>& A)
      : n(A.size()) { // 非負整数列 A から構築
    vector<U> a(n), b(n);
```

wavelet_matrix.cpp (cont.)

```cpp
    for (int i = 0; i < n; i++) {
      if constexpr (is_signed_v<T>)
        assert(A[i] >= 0);
      a[i] = (U)A[i];
    }
    for (int h = LOG - 1; h >= 0; h--) {
      s[h].assign(n + 1, 0);
      for (int i = 0; i < n; i++)
        s[h][i + 1] =
          s[h][i] + (a[i] >> h & 1);
      mid[h] = n - s[h][n];
      int p = 0, q = mid[h];
      for (U x : a)
        ((x >> h & 1) ? b[q++] : b[p++]) =
          x;
      a.swap(b);
    }
  }
  T access(
    int k) const { // 元配列の A[k]を返す
    U x = 0;
    for (int h = LOG - 1; h >= 0; h--) {
      int o = s[h][k], b = s[h][k + 1] - o;
      if (b)
        x |= U(1) << h, k = mid[h] + o;
      else
        k -= o;
    }
    return (T)x;
  }
  int rank(T x, int l,
    int r) const { // [l,r)にある x の個数
    if constexpr (is_signed_v<T>)
      if (x < 0)
        return 0;
    U y = (U)x;
    if constexpr (LOG <
      numeric_limits<U>::digits)
      if (y >> LOG)
        return 0;
    for (int h = LOG - 1; h >= 0; h--) {
      int L = s[h][l], R = s[h][r];
      if (y >> h & 1)
        l = mid[h] + L, r = mid[h] + R;
      else
        l -= L, r -= R;
    }
    return r - l;
  }
  int rank(T x,
    int r) const { // [0,r)にある x の個数
    return rank(x, 0, r);
  }
  T kth(int l, int r, int k)
    const { // [l,r)の k 番目に小さい値 (
  0-indexed)
    U x = 0;
    for (int h = LOG - 1; h >= 0; h--) {
      int L = s[h][l], R = s[h][r],
          z = r - l - (R - L);
      if (k < z)
        l -= L, r -= R;
      else
        x |= U(1) << h, k -= z,
          l = mid[h] + L, r = mid[h] + R;
    }
```

wavelet_matrix.cpp (cont.)

```
    return (T)x;
  }
  T kth_largest(int l, int r,
    int k) const { // [l,r)の k 番目に大きい値
    return kth(l, r, r - l - 1 - k);
  }
  int less(int l, int r, T x)
    const { // [l,r)にある x 未満の値の個数
    if constexpr (is_signed_v<T>)
      if (x <= 0)
        return 0;
    U y = (U)x;
    if constexpr (LOG <
      numeric_limits<U>::digits)
      if (y >> LOG)
        return r - l;
    int c = 0;
    for (int h = LOG - 1; h >= 0; h--) {
      int L = s[h][l], R = s[h][r],
          z = r - l - (R - L);
      if (y >> h & 1)
        c += z, l = mid[h] + L,
                r = mid[h] + R;
      else
        l -= L, r -= R;
    }
    return c;
  }
  int range_freq(int l, int r, T lo, T hi)
    const { // [l,r)で lo<=A[i]<hi の個数
    return less(l, r, hi) - less(l, r, lo);
  }
  optional<T> prev_value(int l, int r,
    T x) const { // [l,r)の x 未満の最大値
    int k = less(l, r, x);
    if (!k)
      return nullopt;
    return kth(l, r, k - 1);
  }
  optional<T> next_value(int l, int r,
    T x) const { // [l,r)の x 以上の最小値
    int k = less(l, r, x);
    if (k == r - l)
      return nullopt;
    return kth(l, r, k);
  }
};
```

z_algorithm.cpp

```
template <class T>
vector<int> z_algorithm(const T& s) {
  int n = s.size();
  vector<int> z(n);
  for (int i = 1, j = 0; i < n; i++) {
    if (i < j + z[j])
      z[i] = min(j + z[j] - i, z[i - j]);
    while (i + z[i] < n &&
      s[z[i]] == s[i + z[i]])
      z[i]++;
    if (i + z[i] > j + z[j])
      j = i;
  }
  if (n)
    z[0] = n;
  return z;
}
```